

Securing **Modbus/TCP** via **SSH Tunneling**

A Cryptographic Mitigation Strategy for Legacy Industrial Embedded Systems

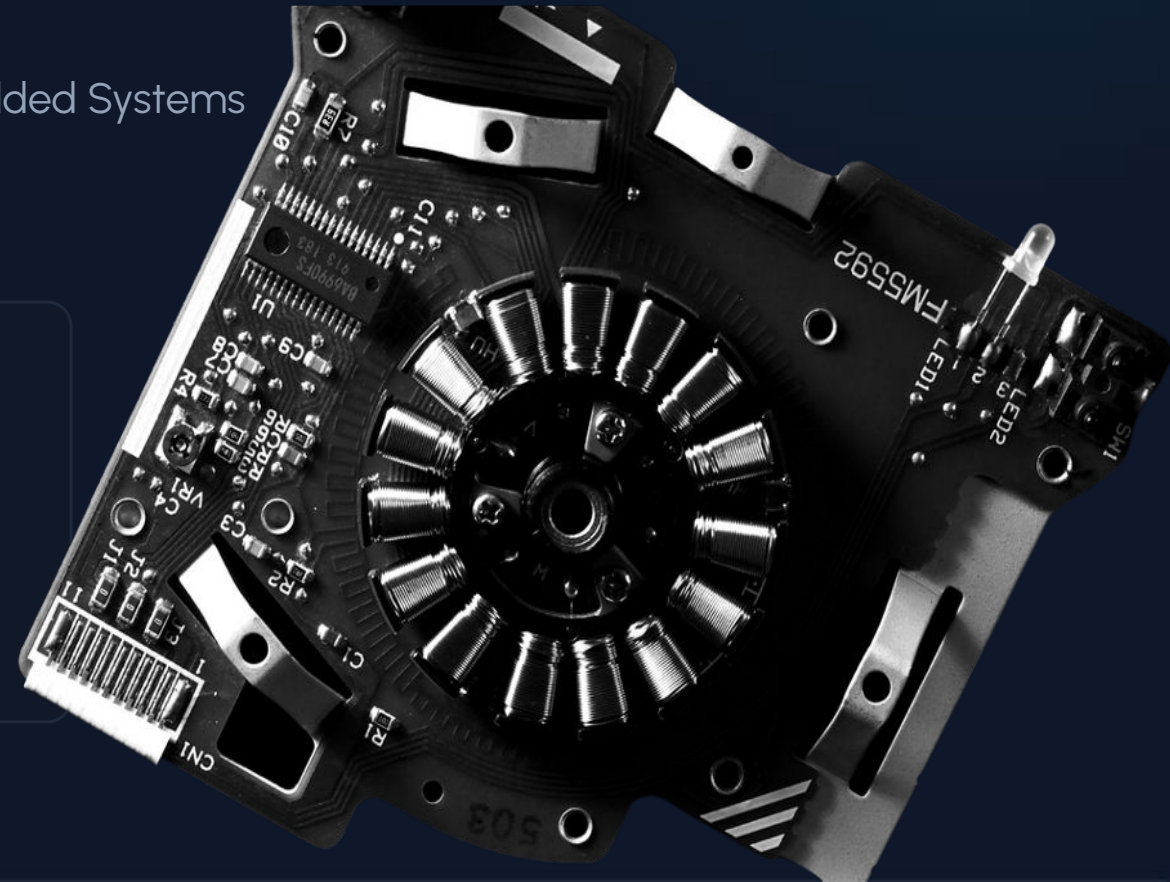
Course Context

Selected Topics of Embedded
Software Development I

Embedded Security SS2026

Project Team

Pavai Vendhan Ganesan
Nandukrishna Menon
Joyal Joshy



Agenda



01	Introduction & Problem Statement	Insecurities of Industrial Control Systems (ICS)
02	Theoretical Background	Modbus/TCP & Threat Landscape
03	Phase I Execution	Plaintext Vulnerability Analysis
04	Cryptographic Mitigation	SSH Tunneling Architecture
05	Phase II Execution	Secure Tunnel Implementation
06	Performance Benchmarking	Feasibility on Embedded Systems
07	Future Trends & Conclusion	-

Introduction to ICS & SCADA

Industrial Control Systems (ICS) and SCADA systems form the digital backbone of modern civilization, regulating power grids, water facilities, and manufacturing floors.

The End of the Air-Gap: Historically, these systems were physically isolated.

Industry 4.0: The Industrial IoT requires real-time data analytics and IT/OT convergence.

The Consequence: This interconnection exposes legacy protocols—designed before modern cyber threats—to the public Internet.



The Modbus/TCP Flaw

The protocol inherently lacks **authentication**, **encryption**, or cryptographic **integrity checks**.

Consequence: High susceptibility to eavesdropping, MITM attacks, and unauthorized data injection.

The Modbus Protocol

Developed in 1979 by Modicon, Modbus is the most widely used industrial protocol. It operates on a strict Master/Slave (Client/Server) architecture.

- 🔧 **Discrete Inputs & Coils:** Used for reading/writing binary sensors and physical actuators (1-bit).
- 🚂 **Input & Holding Registers:** Used for reading/writing analog values (16-bits) like temperature or motor speed.
- 🏠 **Evolution to TCP:** Serial checksums (CRC) were discarded in favor of the TCP/IP stack (Port 502) and an MBAP header.



The Problem & Historical Precedent

Absolute Plaintext

Modbus/TCP transmits all control data and sensor readings in absolute plaintext. It lacks authentication, encryption, or cryptographic integrity checks.

The Stuxnet Context

The 2010 Stuxnet worm demonstrated the catastrophic potential of compromised PLCs. Stuxnet silently modified the control logic of Siemens PLCs governing uranium centrifuges. Lacking integrity checks, the PLCs blindly accepted the malicious logic, resulting in physical destruction.



Threat Landscape for **Modbus/TCP**



Lack of Auth

No username, password, or handshake. Any device reaching Port 502 can read memory or issue commands immediately.



Eavesdropping

Attackers on the local network (e.g., via ARP spoofing) can read all operational data and deduce production recipes.



Data Injection

Lacking cryptographic signatures (MACs), attackers can alter packets in transit or forge entirely new commands.



Replay Attacks

Without sequence numbers, legitimate commands (e.g., "Start Motor") can be captured and maliciously re-transmitted.

Project Architecture

Structured on Clean Architecture, separating industrial logic from the Transport/Security layer.



Virtual PLC (Server)

Uses [pymodbus](#) to instantiate an in-memory PLC with simulated registers.



Modbus Client

Acts as the SCADA terminal, continuously reading and writing data.



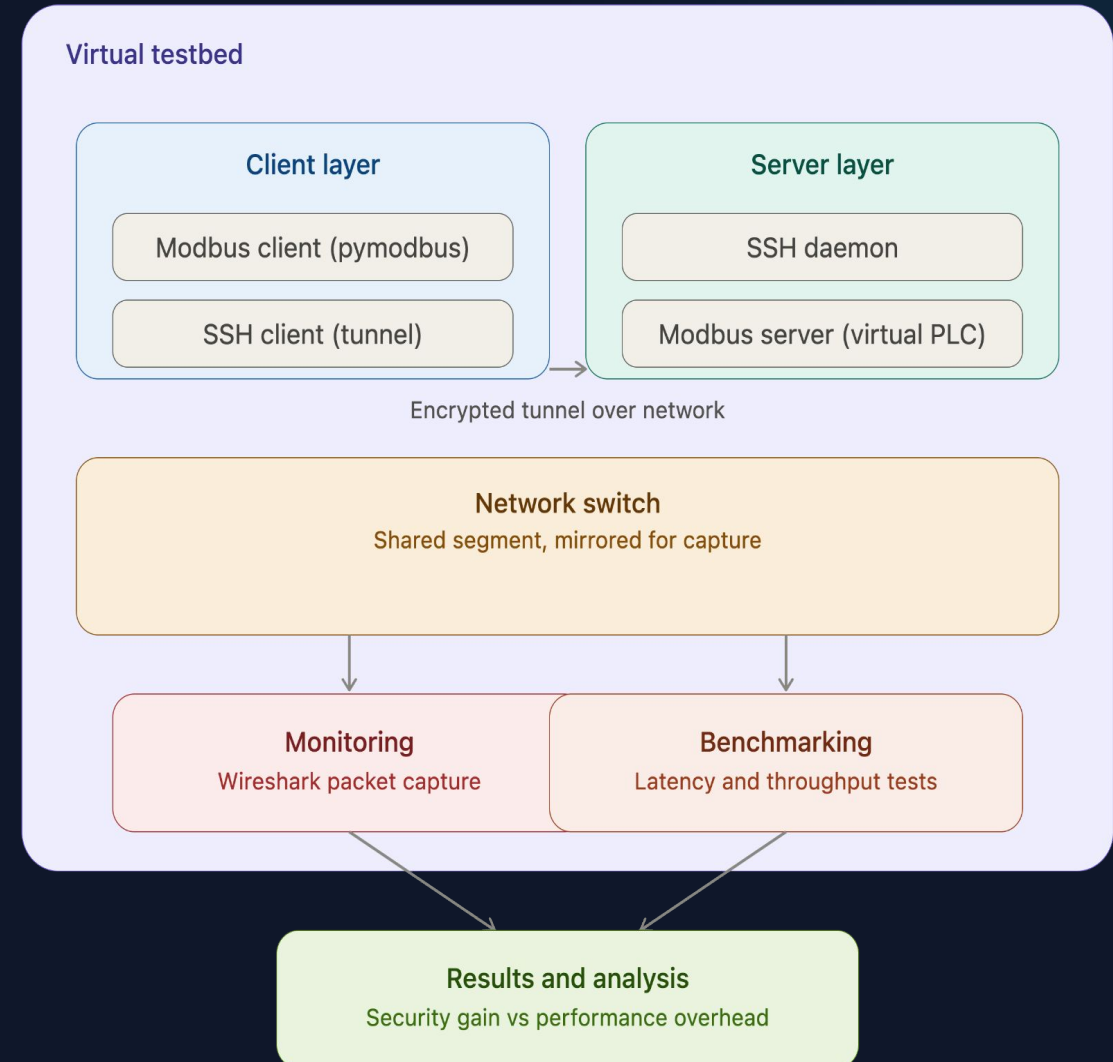
SSH Orchestrator

Dynamically constructs the cryptographic wrapper using the [sshtunnel](#) library.



Makefile Automation

Abstracts Python execution for a unified interface.



Implementation Setup & Workarounds



Port 5020 vs Port 502

UNIX systems prohibit standard users from binding to privileged ports (< 1024). Running the server on Port 502 requires dangerous root access.

Solution: The virtual PLC listens on Port 5020. This circumvents root requirements while preserving exact semantic functionality of Modbus/TCP.



Client-Side Data Generation

A virtual PLC is a "blank brain" without physical sensors. Instead of writing complex background polling threads for the server...

Solution: The Client acts as the simulator, writing simulated Boiler Temps and Motor Speeds to the server, then immediately reading them back. This proves bi-directional encryption later.

Vulnerability Analysis: Base Architecture

Scenario 1: Vulnerable Plaintext

01. System Setup

Direct Connection

A Modbus Client connects directly to a Virtual PLC Server, establishing a standard operational baseline.

02. Clean Architecture

Decoupled Layers

The system layout isolates the core industrial application logic completely from the underlying transport mechanics.

03. Passive Eavesdropping

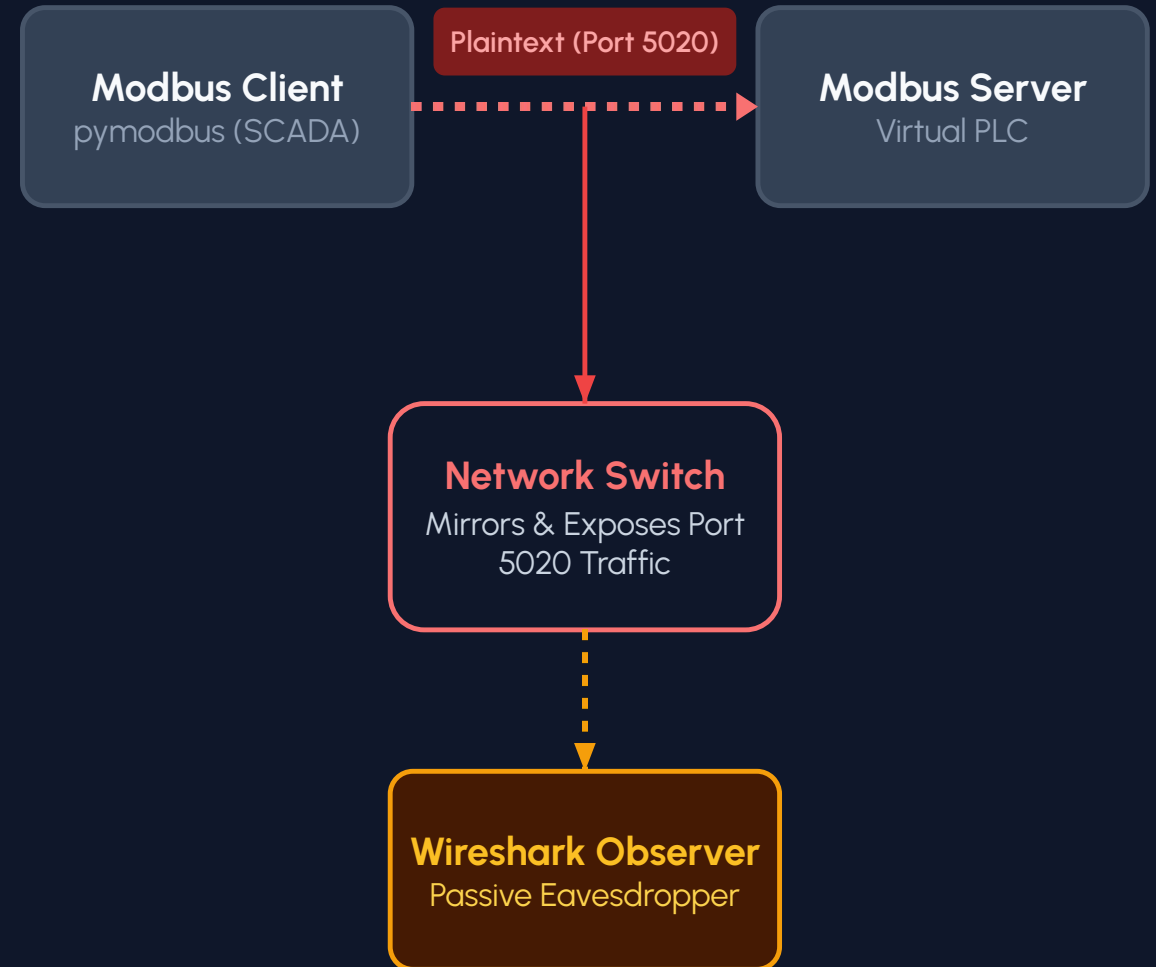
Traffic Mirroring

A network switch transparently duplicates all plaintext traffic flowing on Port 5020 straight to a Wireshark node.

Critical Result

Complete Payload Exposure

Without built-in cryptographic security, industrial control registers and operational parameters remain completely readable by anyone on the network.



Phase I: Plaintext Execution (Base System)

Virtual PLC and Client communicating directly over vulnerable TCP Port 5020.

```
modbus_server.py

Last login: Mon Jun 29 23:48:32 on ttys001
[vedha@MacBookAir ~ % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
[vedha@MacBookAir ESD - Modbus:TCP % make server
python3 src/modbus_server.py
INFO:root:Starting Modbus TCP Server on localhost:5020
INFO:pymodbus.logging:Server listening.
```

```
modbus_client.py

vedha@MacBookAir % make client INFO:root:Connecting to Modbus Server
at 127.0.0.1:5020 INFO:root:Writing Boiler Temp 77°C to register 1
INFO:root:Writing Motor Speed 1489 RPM to register 2 INFO:root:Read
Boiler Temp from register 1: 77°C INFO:root:Read Motor Speed from
register 2: 1489 RPM _____
INFO:root:Writing Boiler Temp 79°C to register 1...
```

Phase I: Eavesdropping Vulnerability (Wireshark)

Capturing traffic on the loopback interface reveals industrial data fully exposed in plaintext.

The image shows a Wireshark packet capture on the loopback interface (lo0). The filter is set to `tcp.port==5020`. The packet list shows a series of Modbus/TCP queries and responses between 127.0.0.1. Packet 27 is selected, showing a Modbus/TCP response for a Read Holding Registers request. The packet details pane shows the following structure:

- > Frame 27: Packet, 67 bytes on wire (536 bits), 67 bytes captured (536 bits) on interface lo0, in Null/Loopback
- > Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
- > Transmission Control Protocol, Src Port: 5020, Dst Port: 52798, Seq: 25, Ack: 37, Len: 11
- > Modbus/TCP
 - Transaction Identifier: 3
 - Protocol Identifier: 0
 - Length: 5
 - Unit Identifier: 0
- > Modbus
 - 0... = Exception: No
 - .000 0011 = Function Code: Read Holding Registers (3)
 - [Request Frame: 25]
 - [Time from request: 1.117000 milliseconds]
 - Byte Count: 2
 - > Register 1 (UINT16): 76

The packet bytes pane shows the raw data in hexadecimal and ASCII:

```
0000 02 00 00 00 45 00 00 3f 00 00 40 00 40 06 00 00  ...E...? ..@...
0010 7f 00 00 01 7f 00 00 01 13 9c ce 3e 6c 22 d6 26  ....>!"&
0020 47 64 9d c0 80 18 18 ec fe 33 00 00 01 01 08 0a  Gd.....3.....
0030 87 65 b2 1e 3f 7f 93 91 00 03 00 00 00 05 00 03  e+7.....
0040 02 00 4c
```

Cryptographic Mitigation: **SSH Tunneling**

Brownfield Remediation

Replacing legacy PLCs is financially prohibitive. We retrofit security using Secure Shell (SSH) Port Forwarding as a transparent wrapper.



Authentication

Identity Verification

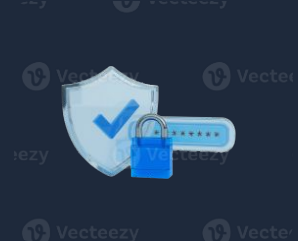
Public/private key pairs mathematically prove identities before data exchange.



Confidentiality

Data Encryption

AES-256-GCM symmetric encryption scrambles the payload into random noise.



Integrity

Message Signing

Hash-based Message Authentication Code (HMAC) appends a signature. Altered bits cause packets to drop.

Secure System Architecture: SSH Tunneled

Scenario 2: SSH Tunneled Process

01 Tunnel Creation

SSH Client establishes encrypted connection to target Daemon.

02 Local Binding

SSH Client opens local port (localhost:2222).

03 Redirection

Modbus Client points to Port 2222 instead of the distant PLC.

04 Encapsulation

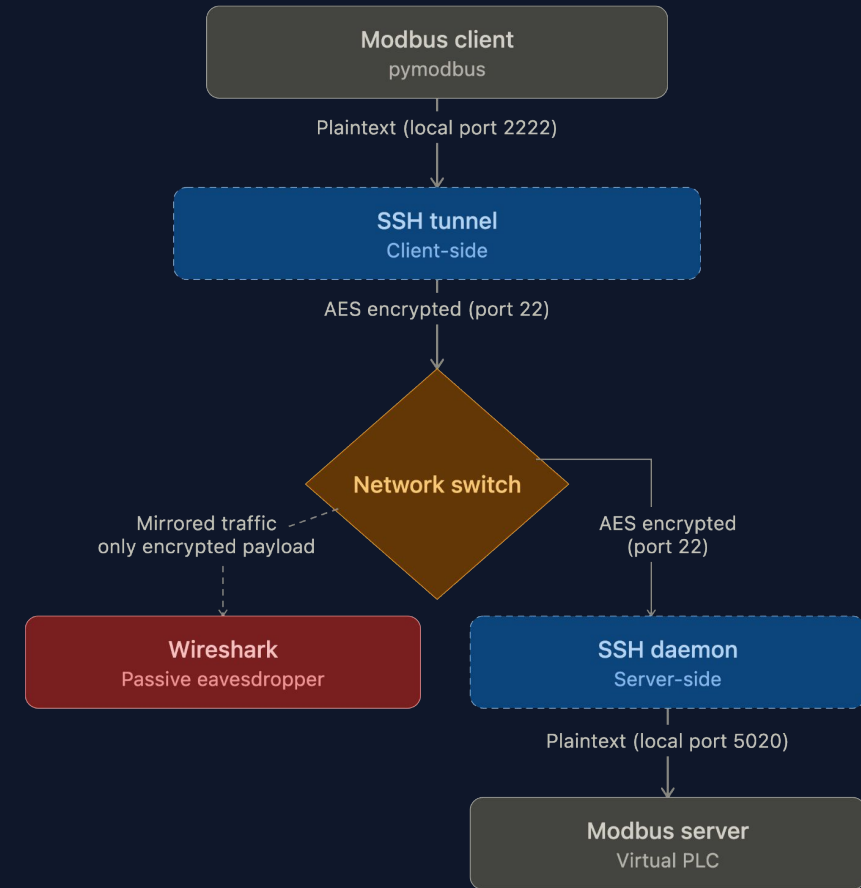
SSH client encrypts traffic via AES and funnels it through Port 22.

05 Decapsulation

Receiving SSH Daemon decrypts and forwards raw bytes to Port 5020.

Result

Application layer remains unaware; encryption occurs transparently at the transport layer.



Result: Modbus payload is protected

The switch and any eavesdropper only see AES ciphertext; register data stays confidential

Phase II: Establishing the Tunnel

The project supports establishing the cryptographic wrapper via native OS tools or a Python library.

Option A: Native macOS SSH

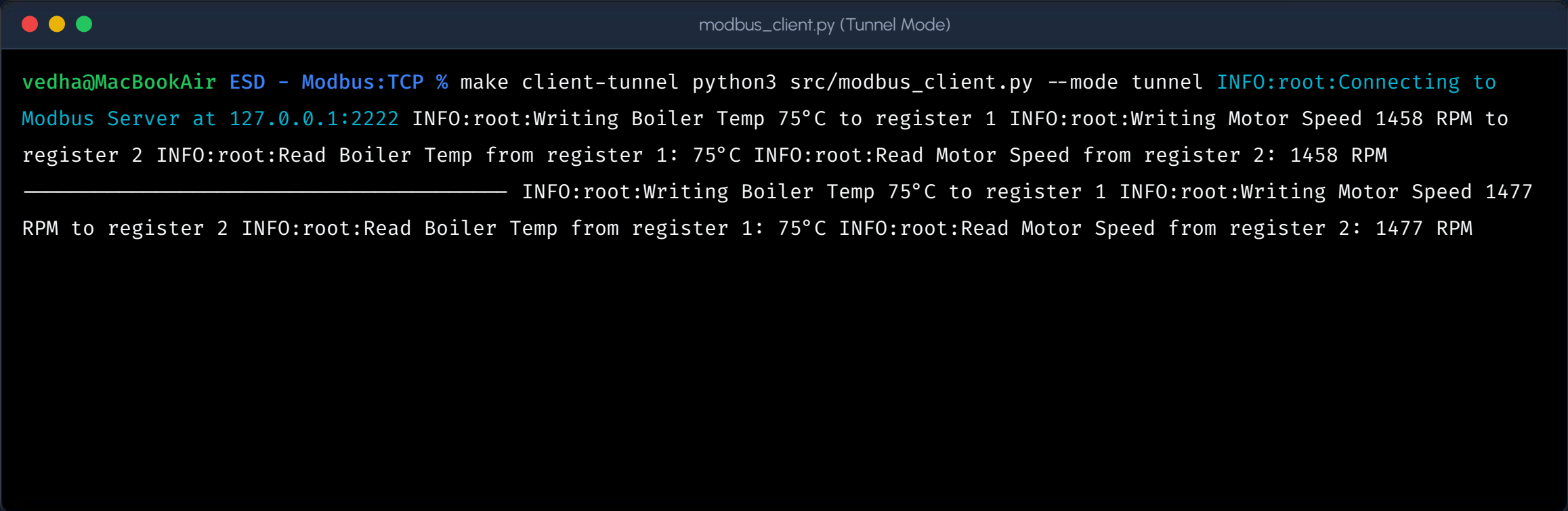
```
vedha@MacBookAir % make tunnel-native Starting Native SSH
Tunnel (Inbuild) on port 2222 ... ssh -N -L 2222:127.0.0.1:5020
$(whoami)@127.0.0.1 _
```

Option B: Python sshtunnel

```
vedha@MacBookAir % make tunnel-python == Modbus/TCP SSH Tunnel
Setup == Enter your macOS username: vedha Enter your macOS
password: [+] SUCCESS! SSH Tunnel established. Local Entrance:
Port 2222 Remote Destination: Port 5020
```

Phase II: Secure Client Execution

The Modbus client is dynamically re-configured to point its TCP socket to the new local endpoint (Port 2222). Application logic executes flawlessly.

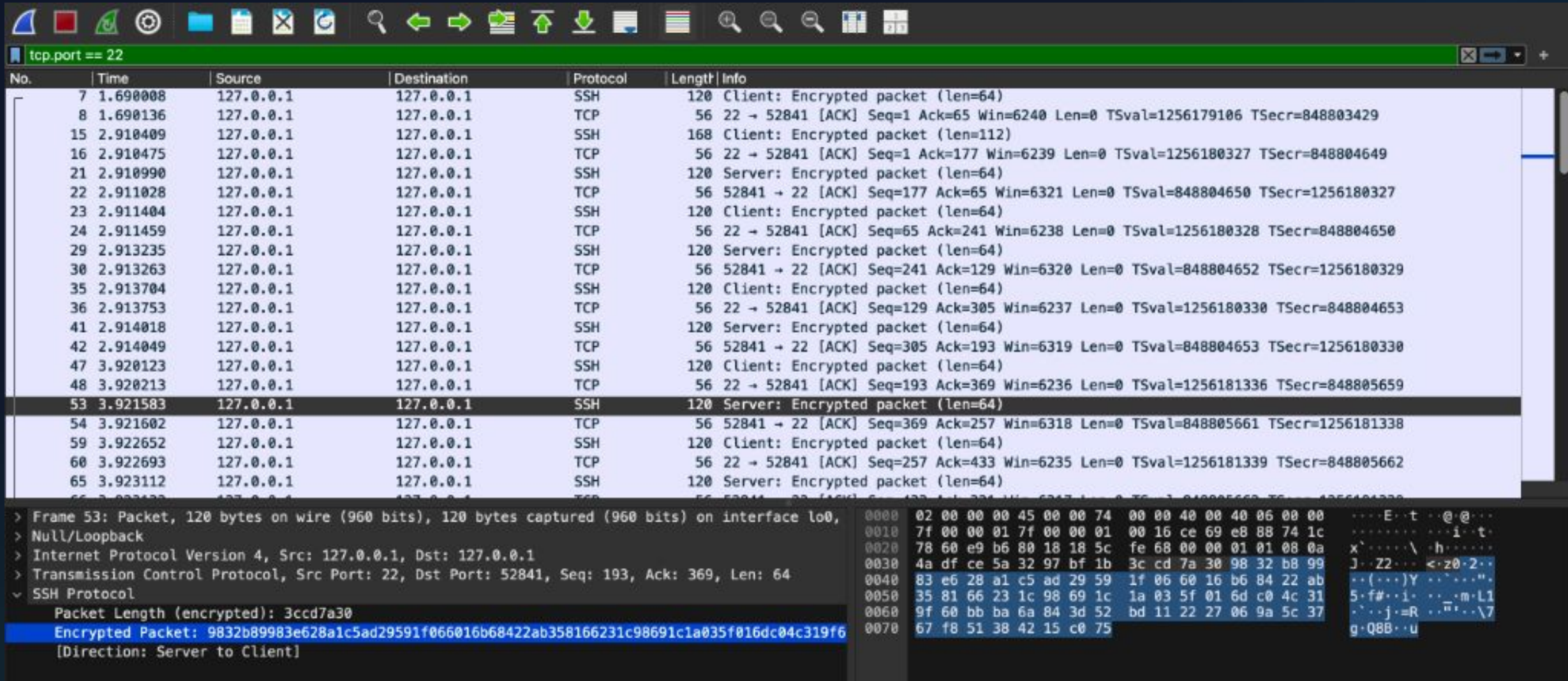


```
modbus_client.py (Tunnel Mode)

vedha@MacBookAir ESD - Modbus:TCP % make client-tunnel python3 src/modbus_client.py --mode tunnel INFO:root:Connecting to
Modbus Server at 127.0.0.1:2222 INFO:root:Writing Boiler Temp 75°C to register 1 INFO:root:Writing Motor Speed 1458 RPM to
register 2 INFO:root:Read Boiler Temp from register 1: 75°C INFO:root:Read Motor Speed from register 2: 1458 RPM
INFO:root:Writing Boiler Temp 75°C to register 1 INFO:root:Writing Motor Speed 1477
RPM to register 2 INFO:root:Read Boiler Temp from register 1: 75°C INFO:root:Read Motor Speed from register 2: 1477 RPM
```


Phase II: Blinding the Attacker (Wireshark)

Capturing traffic on the loopback interface reveals industrial data is secure by mathematically random AES sequences. Confidentiality and Integrity are achieved.



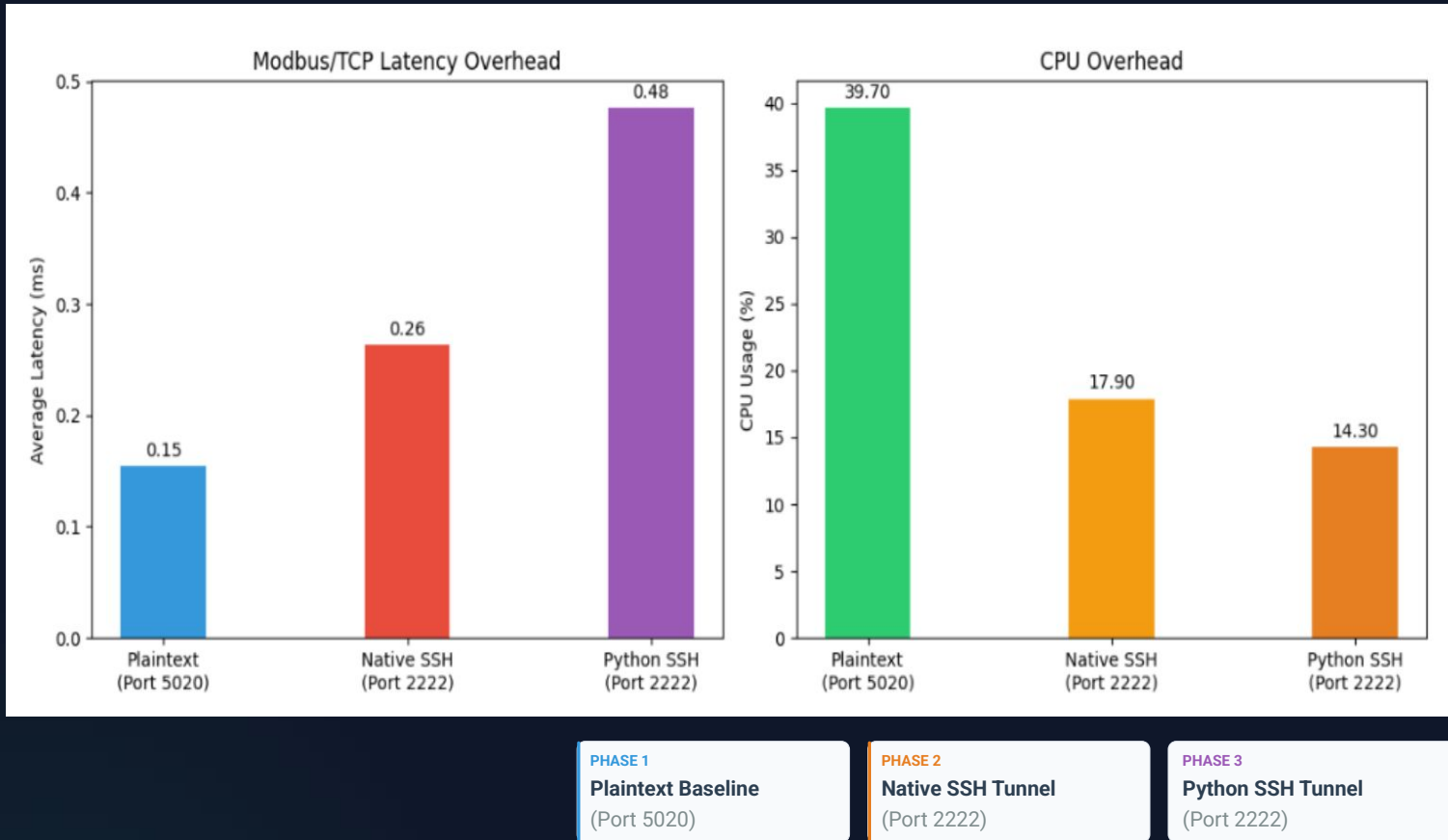
tcp.port == 22

No.	Time	Source	Destination	Protocol	Length	Info
7	1.690008	127.0.0.1	127.0.0.1	SSH	120	Client: Encrypted packet (len=64)
8	1.690136	127.0.0.1	127.0.0.1	TCP	56	22 → 52841 [ACK] Seq=1 Ack=65 Win=6240 Len=0 TSval=1256179106 TSecr=848803429
15	2.910409	127.0.0.1	127.0.0.1	SSH	168	Client: Encrypted packet (len=112)
16	2.910475	127.0.0.1	127.0.0.1	TCP	56	22 → 52841 [ACK] Seq=1 Ack=177 Win=6239 Len=0 TSval=1256180327 TSecr=848804649
21	2.910990	127.0.0.1	127.0.0.1	SSH	120	Server: Encrypted packet (len=64)
22	2.911028	127.0.0.1	127.0.0.1	TCP	56	52841 → 22 [ACK] Seq=177 Ack=65 Win=6321 Len=0 TSval=848804650 TSecr=1256180327
23	2.911404	127.0.0.1	127.0.0.1	SSH	120	Client: Encrypted packet (len=64)
24	2.911459	127.0.0.1	127.0.0.1	TCP	56	22 → 52841 [ACK] Seq=65 Ack=241 Win=6238 Len=0 TSval=1256180328 TSecr=848804650
29	2.913235	127.0.0.1	127.0.0.1	SSH	120	Server: Encrypted packet (len=64)
30	2.913263	127.0.0.1	127.0.0.1	TCP	56	52841 → 22 [ACK] Seq=241 Ack=129 Win=6320 Len=0 TSval=848804652 TSecr=1256180329
35	2.913704	127.0.0.1	127.0.0.1	SSH	120	Client: Encrypted packet (len=64)
36	2.913753	127.0.0.1	127.0.0.1	TCP	56	22 → 52841 [ACK] Seq=129 Ack=305 Win=6237 Len=0 TSval=1256180330 TSecr=848804653
41	2.914018	127.0.0.1	127.0.0.1	SSH	120	Server: Encrypted packet (len=64)
42	2.914049	127.0.0.1	127.0.0.1	TCP	56	52841 → 22 [ACK] Seq=305 Ack=193 Win=6319 Len=0 TSval=848804653 TSecr=1256180330
47	3.920123	127.0.0.1	127.0.0.1	SSH	120	Client: Encrypted packet (len=64)
48	3.920213	127.0.0.1	127.0.0.1	TCP	56	22 → 52841 [ACK] Seq=193 Ack=369 Win=6236 Len=0 TSval=1256181336 TSecr=848805659
53	3.921583	127.0.0.1	127.0.0.1	SSH	120	Server: Encrypted packet (len=64)
54	3.921602	127.0.0.1	127.0.0.1	TCP	56	52841 → 22 [ACK] Seq=369 Ack=257 Win=6318 Len=0 TSval=848805661 TSecr=1256181338
59	3.922652	127.0.0.1	127.0.0.1	SSH	120	Client: Encrypted packet (len=64)
60	3.922693	127.0.0.1	127.0.0.1	TCP	56	22 → 52841 [ACK] Seq=257 Ack=433 Win=6235 Len=0 TSval=1256181339 TSecr=848805662
65	3.923112	127.0.0.1	127.0.0.1	SSH	120	Server: Encrypted packet (len=64)

> Frame 53: Packet, 120 bytes on wire (960 bits), 120 bytes captured (960 bits) on interface lo0,
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 22, Dst Port: 52841, Seq: 193, Ack: 369, Len: 64
SSH Protocol
Packet Length (encrypted): 3ccd7a30
Encrypted Packet: 9832b89983e628a1c5ad29591f066016b68422ab358166231c98691c1a035f016dc04c319f6
[Direction: Server to Client]

0000 02 00 00 00 45 00 00 74 00 00 40 00 40 06 00 00E..t...@..@..
0010 7f 00 00 01 7f 00 00 01 00 16 ce 69 e8 88 74 1ci..t..
0020 78 60 e9 b6 80 18 18 5c fe 68 00 00 01 01 08 0a x'.....\..h.....
0030 4a df ce 5a 32 97 bf 1b 3c cd 7a 30 98 32 b8 99 J..22...<.20.2..
0040 83 e6 28 a1 c5 ad 29 59 1f 06 60 16 b6 84 22 ab ..{(...).Y...".
0050 35 81 66 23 1c 98 69 1c 1a 03 5f 01 6d c0 4c 31 5.f#...i...m.L1
0060 9f 60 bb ba 6a 84 3d 52 bd 11 22 27 06 9a 5c 37 .'.j..=R...n...7
0070 67 f8 51 38 42 15 c0 75 g.Q8B..u

Feasibility: Performance Benchmarking



Methodology

Automated Python suite executes 1,000 rapid, consecutive Modbus read requests across three phases to evaluate computational cost on embedded systems.

```
ESD - Modbus/TCP — -zsh — 142x27

vedha@MacBookAir ESD - Modbus:TCP % cd "/Users/vedha/Documents/ESD - Modbus:TCP"
vedha@MacBookAir ESD - Modbus:TCP % make benchmark
python3 src/modbus_benchmark.py
=== Phase 1: Benchmarking Plaintext Modbus (Port 5020) ===
Executing 1000 consecutive read requests...
-> Avg Latency: 0.1541 ms
-> CPU Usage: 39.70%

=== Phase 2: Benchmarking Native SSH Tunnel (Port 2222) ===
Please open a new terminal and run: ssh -N -L 2222:127.0.0.1:5020 $(whoami)@127.0.0.1
Press Enter once the native SSH tunnel is running...
Executing 1000 consecutive read requests...
-> Avg Latency: 0.2637 ms
-> CPU Usage: 17.90%

=== Phase 3: Benchmarking Python SSH Tunnel (Port 2222) ===
Please KILL the native SSH tunnel (Ctrl+C), then run: make tunnel-python
Press Enter once the Python SSH tunnel is running...
Executing 1000 consecutive read requests...
-> Avg Latency: 0.4766 ms
-> CPU Usage: 14.30%

=== Generating Visualization Graphs ===

[+] Success! Graph saved to 'docs/images/benchmark_results.png'
vedha@MacBookAir ESD - Modbus:TCP %
```


Future Trends in OT Security

SSH tunneling is an excellent brownfield mitigation, but the future of industrial embedded systems lies in secure-by-design architectures.

- ➡ **Protocol Migration:** Shift to OPC UA and MQTT, which integrate TLS encryption natively.
- 🔒 **Zero Trust Architecture (ZTA):** "Never Trust, Always Verify" - requiring cryptographic identity even on isolated VLANs.
- 🔧 **Hardware Security (HSM/TPM):** Silicon-level protection for cryptographic private keys.
- 🤖 **AI-Powered IDS:** Machine learning algorithms on switches to detect anomalous Modbus behaviors.



Conclusion

The Reality: While migrating to natively secure protocols is ideal, legacy infrastructure will remain for decades. Transparent wrappers are an absolute necessity.

KEY VULNERABILITY

Modbus/TCP is critically flawed for interconnected environments.

THE RETROFIT SOLUTION

SSH Tunneling successfully retrofits Confidentiality, Integrity, and Authentication transparently.

FEASIBILITY METRICS

Performance metrics prove it is a highly feasible, cost-effective stopgap for legacy infrastructure.

Questions?



SOURCE CODE REPOSITORY

github.com/pavaivendhan/ESD-Modbus-SSH-Tunnel